

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Week #11: Program Debugging



Program Debugging Using gdb



- A debugging tool for C, C++
- GDB allows to pause program execution at specified points
- Allows to execute program line by line
- Inspect the values in the variables
- Step through code line by line or function by function
- Help in localizing errors and identifying errors in programs

Compiling for debugging & execution



- `gcc random.c // a.out` executable without debugging information
- `gcc random.c -g // a.out` executable with debugging information
- `gdb` is an executable which takes another executable as input to debug the program
- `gdb a.out`
- `gdb >`



Logging debugging information

- Record the commands & output of the gdb during program execution
- `set logging on` // enables logging
- `set logging off` // disables logging
- `set logging file <file_name>` // write the gdb output to file_name
- Default file in which logging information recorded is `gdb.txt`
- Example contents of logging as shown.



Logging debugging information

```
1 1 #include<stdio.h>
2 2 #include<stdlib.h>
3 3
4 4 int main(int argc, char *argv[])
5 5 {
6 6     int n = rand() % 100;
7 7     int i = 0;
8 8
9 9     int data[n];
10 10     for( i = 0; i < n; i++ )
11 Breakpoint 1 at 0x11db: file rand.c, line 6.
12 Starting program: /home/saradhi/courses/cs204/july-dec-2024/labs/week11/a.o
13 Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
14 ^M[###
15 Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
16
17 Breakpoint 1, main (argc=1, argv=0x7fffffffde38) at rand.c:6
18 6     int n = rand() % 100;
19 7     int i = 0;
20 $1 = 83
21 9     int data[n];
22 $2 = 0
```

Logging debugging information



```
23 10     for( i = 0; i < n; i++ )
24 5 {
25 6     int n = rand() % 100;
26 7     int i = 0;
27 8
28 9     int data[n];
29 10     for( i = 0; i < n; i++ )
30 11     {
31 12         data[i] = rand() % 100;
32 13         printf("insert %3d\n", data[i]);
33 14     }
34 12         data[i] = rand() % 100;
35 13         printf("insert %3d\n", data[i]);
36 10     for( i = 0; i < n; i++ )
37 12         data[i] = rand() % 100;
38 13         printf("insert %3d\n", data[i]);
39 10     for( i = 0; i < n; i++ )
40 12         data[i] = rand() % 100;
41 13         printf("insert %3d\n", data[i]);
42 10     for( i = 0; i < n; i++ )
43 12         data[i] = rand() % 100;
44 13         printf("insert %3d\n", data[i]);
```

Logging debugging information



```
45 10     for( i = 0; i < n; i++ )
46 12         data[i] = rand() % 100;
47 13         printf("insert %3d\n", data[i]);
48 10     for( i = 0; i < n; i++ )
49 $3 = 4
50 $4 = 83
51 5 {
52 6     int n = rand() % 100;
53 7     int i = 0;
54 8
55 9     int data[n];
56 10     for( i = 0; i < n; i++ )
57 11     {
58 12         data[i] = rand() % 100;
59 13         printf("insert %3d\n", data[i]);
60 14     }
61 15     printf("print\n");
62 16
63 17     for( i = 0; i < n; i++ )
64 18     {
65 19         printf("delete %3d\n", data[i]);
66 20     }
```



~~~~~

# Important commands



- **Breakpoints**
- **Watchpoints**
- **Catchpoints**
- **Inspect variables**



# Common commands

- **run** – starts program execution
- **break** – set a break point where program execution must stop
- **continue** – instruct gdb to continue from the current point
- **next** – steps to next line (skips function calls)
- **step** – step into functions
- **print** – print value in specified variable



# Live Demonstration





**Thank You**





**Questions?**

